u-1

p-1 hashing

✍️ **DESCRIPTION (Exam Format)**

A **Hash Table** is a data structure used to store data efficiently by mapping keys to specific indices using a **hash function**. It allows fast insertion, searching, and deletion operations.

A commonly used hash function is:

h(key) = key % table_size

Using this function, a key is converted into an index where the data is stored in the hash table.

However, sometimes multiple keys may map to the same index. This situation is called a **collision**. To handle collisions, different collision resolution techniques are used.

Separate Chaining is a technique in which each index of the hash table contains a linked list. When multiple keys hash to the same index, they are stored as nodes in the linked list.

Linear Probing is a method of open addressing where, if a collision occurs, the next available slot is searched sequentially until an empty position is found.

Quadratic Probing is another open addressing technique where the position of the next slot is determined using a quadratic function, reducing clustering compared to linear probing.

These collision resolution techniques ensure efficient utilization of the hash table and maintain the performance of operations like insertion, searching, and deletion.

CODE:

```
#include<stdio.h>

#include<stdlib.h>

#include<limits.h>


#define TABLE_SIZE 11

#define EMPTY_SLOT INT_MIN

#define DELETED_SLOT (INT_MIN+1)


/* ----------- CHAINING ----------- */


typedef struct Node{

    int key;
```

```
    struct Node* next;
} Node;

Node* chain_table[TABLE_SIZE];
void init_chaining(){
    for(int i=0;i<TABLE_SIZE;i++)
        chain_table[i]=NULL;
}

int hash_fn(int key){
    int h = key % TABLE_SIZE;
    if(h < 0) h += TABLE_SIZE;
    return h;
}

void chaining_insert(int key){
    int idx = hash_fn(key);
    Node* newnode = (Node*)malloc(sizeof(Node));

    if(!newnode){
        printf("Memory error\n");
        return;
    }

    newnode->key = key;
    newnode->next = chain_table[idx];
    chain_table[idx] = newnode;

    printf("Inserted %d at chain index %d\n",key,idx);
```

```
}

Node* chaining_search_node(int key){
    int idx = hash_fn(key);
    Node* cur = chain_table[idx];

    while(cur){
        if(cur->key == key){
            return cur;
        }
        cur = cur->next;
    }
    return NULL;
}
void chaining_delete(int key){
    int idx = hash_fn(key);
    Node *cur = chain_table[idx];
    Node *prev = NULL;

    while(cur){
        if(cur->key == key){
            if(prev == NULL)
                chain_table[idx] = cur->next;
            else
                prev->next = cur->next;

            free(cur);
            printf("Deleted\n");
            return;
```

```
        }
        prev = cur;
        cur = cur->next;
    }
    printf("Not found\n");
}

void chaining_display(){
    for(int i=0;i<TABLE_SIZE;i++){
        printf("[%2d]:",i);
        Node* cur = chain_table[i];
        while(cur){
            printf(" -> %d",cur->key);
            cur = cur->next;
        }
        printf(" -> NULL\n");
    }
}

/* ----------- OPEN ADDRESSING ----------- */

int oa_table[TABLE_SIZE];

void init_open_addressing(){
    for(int i=0;i<TABLE_SIZE;i++)
        oa_table[i] = EMPTY_SLOT;
}

/* ----------- LINEAR PROBING ----------- */
```

```
int linear_insert(int key){
    int idx = hash_fn(key);

    for(int i=0;i<TABLE_SIZE;i++){
        int pos = (idx + i) % TABLE_SIZE;

        if(oa_table[pos]==EMPTY_SLOT || oa_table[pos]==DELETED_SLOT){
            oa_table[pos] = key;
            printf("Inserted %d at index %d (linear)\n",key,pos);
            return 1;
        }
    }
    printf("Table full (linear)\n");
    return 0;
}

int linear_search(int key){
    int idx = hash_fn(key);

    for(int i=0;i<TABLE_SIZE;i++){
        int pos = (idx + i) % TABLE_SIZE;

        if(oa_table[pos]==EMPTY_SLOT)
            return -1;

        if(oa_table[pos]==key)
            return pos;
    }
```

```
    return -1;
}

int linear_delete(int key){
    int pos = linear_search(key);

    if(pos==-1){
        printf("%d not found (linear)\n",key);
        return 0;
    }

    oa_table[pos] = DELETED_SLOT;
    printf("Deleted %d from index %d (linear)\n",key,pos);
    return 1;
}

/* ----------- QUADRATIC PROBING ----------- */

int quadratic_insert(int key){
    int idx = hash_fn(key);

    for(int i=0;i<TABLE_SIZE;i++){
        int pos = (idx + i*i) % TABLE_SIZE;

        if(oa_table[pos]==EMPTY_SLOT || oa_table[pos]==DELETED_SLOT){
            oa_table[pos] = key;
            printf("Inserted %d at index %d (quadratic)\n",key,pos);
            return 1;
        }
```

```
    }
    printf("Table full (quadratic)\n");
    return 0;
}

int quadratic_search(int key){
    int idx = hash_fn(key);

    for(int i=0;i<TABLE_SIZE;i++){
        int pos = (idx + i*i) % TABLE_SIZE;

        if(oa_table[pos]==EMPTY_SLOT)
            return -1;

        if(oa_table[pos]==key)
            return pos;
    }
    return -1;
}

int quadratic_delete(int key){
    int pos = quadratic_search(key);

    if(pos==-1){
        printf("%d not found (quadratic)\n",key);
        return 0;
    }

    oa_table[pos] = DELETED_SLOT;
```

```
    printf("Deleted %d from index %d (quadratic)\n",key,pos);
    return 1;
}

/* ----------- DISPLAY ----------- */

void open_display(){
    printf("Index : Value\n");

    for(int i=0;i<TABLE_SIZE;i++){
        if(oa_table[i]==EMPTY_SLOT)
            printf("[%2d] : EMPTY\n",i);
        else if(oa_table[i]==DELETED_SLOT)
            printf("[%2d] : DELETED\n",i);
        else
            printf("[%2d] : %d\n",i,oa_table[i]);
    }
}

/* ----------- MAIN ----------- */

int main(){
    int choice, key;

    while(1){
        printf("\n1.Chaining 2.Linear 3.Quadratic 0.Exit\n");
        scanf("%d",&choice);

        if(choice==0) break;
```

```
if(choice==1){
    init_chaining();

    while(1){
        printf("\n1.Insert 2.Search 3.Delete 4.Display 0.Back\n");
        int ch; scanf("%d",&ch);

        if(ch==0) break;

        if(ch==1){
            scanf("%d",&key);
            chaining_insert(key);
        }
        else if(ch==2){
            scanf("%d",&key);
            if(chaining_search_node(key,NULL))
                printf("Found\n");
            else
                printf("Not Found\n");
        }
        else if(ch==3){
            scanf("%d",&key);
            chaining_delete(key);
        }
        else if(ch==4){
            chaining_display();
        }
    }
```

```
}

else if(choice==2){
    init_open_addressing();

    while(1){
        printf("\n1.Insert 2.Search 3.Delete 4.Display 0.Back\n");
        int ch; scanf("%d",&ch);

        if(ch==0) break;

        if(ch==1){
            scanf("%d",&key);
            linear_insert(key);
        }
        else if(ch==2){
            scanf("%d",&key);
            int pos = linear_search(key);
            if(pos==-1) printf("Not Found\n");
            else printf("Found at %d\n",pos);
        }
        else if(ch==3){
            scanf("%d",&key);
            linear_delete(key);
        }
        else if(ch==4){
            open_display();
        }
    }
```

```
}

else if(choice==3){
    init_open_addressing();

    while(1){
        printf("\n1.Insert 2.Search 3.Delete 4.Display 0.Back\n");
        int ch; scanf("%d",&ch);

        if(ch==0) break;

        if(ch==1){
            scanf("%d",&key);
            quadratic_insert(key);
        }
        else if(ch==2){
            scanf("%d",&key);
            int pos = quadratic_search(key);
            if(pos==-1) printf("Not Found\n");
            else printf("Found at %d\n",pos);
        }
        else if(ch==3){
            scanf("%d",&key);
            quadratic_delete(key);
        }
        else if(ch==4){
            open_display();
        }
    }
```

```
        }
    }

    return 0;
}
```

🔥 **MOST IMPORTANT LINE (REMEMBER)**

```
newnode->next = chain_table[idx];
```

👉 This is what **creates chaining**

```
#include<stdio.h>
#include<stdlib.h>

// Node structure
struct node {
    int key;
    struct node *left, *right;
    int height;
};

// Utility functions
int max(int a, int b) {
    return (a > b) ? a : b;
}

int height(struct node *n) {
    if(n == NULL) return 0;
    return n->height;
```

```
}

// Create new node
struct node* newnode(int key) {
    struct node* node = (struct node*)malloc(sizeof(struct node));
    node->key = key;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}

// Rotations
struct node* rightRotate(struct node *y) {
    struct node *x = y->left;
    struct node *T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

struct node* leftRotate(struct node *x) {
    struct node *y = x->right;
    struct node *T2 = y->left;
```

```
    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

// Get balance factor
int getBalance(struct node *node) {
    if(node == NULL) return 0;
    return height(node->left) - height(node->right);
}

// Insert
struct node* insertNode(struct node *node, int key) {

    // Normal BST insert
    if(node == NULL) {
        printf("Node %d inserted\n", key);
        return newnode(key);
    }

    if(key < node->key)
        node->left = insertNode(node->left, key);
    else if(key > node->key)
        node->right = insertNode(node->right, key);
    else {
```

```
    printf("Duplicate not allowed\n");
    return node;
}

// Update height
node->height = 1 + max(height(node->left), height(node->right));

// Get balance
int balance = getBalance(node);

// Rotations
// LL
if(balance > 1 && key < node->left->key)
    return rightRotate(node);

// RR
if(balance < -1 && key > node->right->key)
    return leftRotate(node);

// LR
if(balance > 1 && key > node->left->key) {
    node->left = leftRotate(node->left);
    return rightRotate(node);
}

// RL
if(balance < -1 && key < node->right->key) {
    node->right = rightRotate(node->right);
    return leftRotate(node);
```

```
    }

    return node;
}

// Search
struct node* search(struct node *root, int key) {
    if(root == NULL || root->key == key)
        return root;

    if(key < root->key)
        return search(root->left, key);

    return search(root->right, key);
}

// Traversals
void inorder(struct node *root) {
    if(root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}

void preorder(struct node *root) {
    if(root != NULL) {
        printf("%d ", root->key);
        preorder(root->left);
```

```
        preorder(root->right);
    }
}

void postorder(struct node *root) {
    if(root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->key);
    }
}

// Main
int main() {
    struct node *root = NULL;
    int choice, key;

    while(1) {
        printf("\n--- AVL TREE MENU ---");
        printf("\n1.Insert\n2.Search\n3.Inorder\n4.Preorder\n5.Postorder\n6.Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);

        switch(choice) {

            case 1:
                printf("Enter key: ");
                scanf("%d", &key);
                root = insertNode(root, key);
```

```
    break;

case 2:
    printf("Enter key to search: ");
    scanf("%d", &key);
    if(search(root, key))
        printf("FOUND\n");
    else
        printf("NOT FOUND\n");
    break;

case 3:
    printf("Inorder: ");
    inorder(root);
    break;

case 4:
    printf("Preorder: ");
    preorder(root);
    break;

case 5:
    printf("Postorder: ");
    postorder(root);
    break;

case 6:
    exit(0);
```

```
            default:
                printf("Invalid choice\n");
        }
    }
}
```